

Testing

Tests live beside the code, in the same file or in `tests/`.

```
use std.io
use std.testing

fn double(n: Int) -> Int => n * 2

test "double doubles" {
  testing.same(double(2), 4, "two")
  testing.same(double(0), 0, "zero")
  testing.same(double(0 - 3), 0 - 6, "negative")
}

fn main() {
  io.say("{double(21)}")
}
```

Run them with:

```
sprfst test
```

Every `test` block runs on its own, and a failure reports which assertion gave way and what it saw instead.

What `std.testing` offers

- `same(got, wanted, what)` —fails unless they match
- `different(got, unwanted, what)`
- `yes(condition, what)` and `no(condition, what)`
- `close(got, wanted, tolerance, what)` —for numbers with decimals
- `holds(list, item, what)`
- `failed(result, what)` —expects an `Err`

```
use std.testing
use std.io

fn average(xs: [Num]) -> Num {
  if xs.is_empty() { give 0.0 }
  give xs.sum() / xs.len()
}

test "average" {
  testing.close(average([2.0, 4.0]), 3.0, 0.0001, "two numbers")
  testing.close(average([]), 0.0, 0.0001, "no numbers")
}

fn main() { io.say("{average([1.0, 2.0, 3.0])}") }
```

Benchmarks

`bench` blocks are timed rather than checked. The runner repeats the body until it has a steady figure and reports operations per second.

```
use std.io

bench "building a list" {
    var xs: [Int] = []
    var i = 0
    while i < 500 {
        xs.push(i)
        i = i + 1
    }
}

fn main() { io.say("run me with sprfst test") }
```

Exercise

Take the palindrome function from chapter six and write tests for the empty string, a single letter, a real palindrome and a non-palindrome.